

Lab 1 C++: Building Quantum Circuits for Real Hardware

A walkthrough for QGSS 2026 participants

James Weaver, July 2026, Qiskit Global Summer School

Where we are

- **Lab 0:** you installed Qiskit, set up your IBM Quantum account, and ran your first quantum program
- **Lab 1** (today, C++ version): build real circuits in C++ using the qiskit-cpp SDK, understand circuit depth, and prep for hardware
- **Labs 2-4:** apply what you learn here to bigger problems

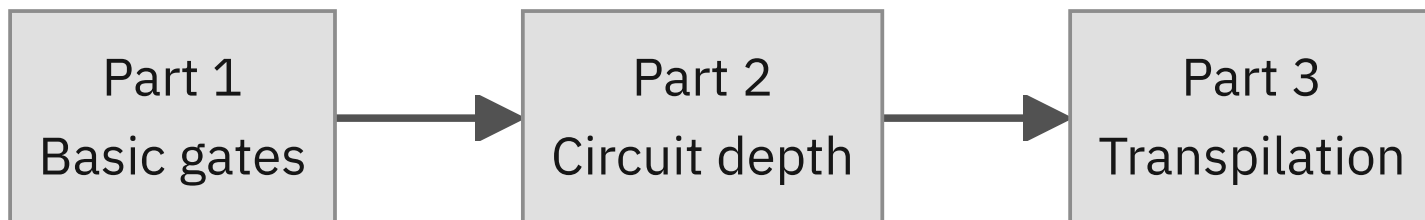
If you finished Lab 0 and you know what $H|0\rangle$ does, you're ready.

What you'll be able to do by the end

- Build **Bell states** and **GHZ states** in C++
- Reason about **circuit depth** and why it matters for noise
- Adapt circuits to a real chip's **connectivity constraints**
- Grade your work via the same QGSS grader the Python lab uses

7 exercises. Each one has an auto-grader. Take your time. Some exercises are quick, others reward careful thinking.

How the lab is organized



Each part builds on the previous. By Part 3 you'll be designing 64-qubit circuits.

Getting started on qBraid

This lab is supported on **qBraid**, where the C++ lab files, `qiskit-cpp`, and the build tools are already provided in your environment.

1. Open the lab in **qBraid Lab** and select the QGSS environment/kernel.
2. Open `cpp_lab_1.ipynb`.
3. Run the **Setup** cells top to bottom — they locate `LAB_ROOT / qiskit-cpp`, install the Python deps, and configure CMake.
4. Work through the parts in order (**Runtime** → **Run All** also works).

Everything runs locally on qBraid against the `FakeTorino` simulator — **no IBM Quantum runtime minutes** are consumed. (Submitting answers to the grader needs a configured IBM Quantum account; see the Setup section.)

Part 1: Basic Gates

James Weaver, July 2026, Qiskit Global Summer School

6 / 29

Three gates do most of the work

GATE	WHAT IT DOES	NOTATION
X	Flips a qubit: $ 0\rangle \rightarrow 1\rangle$ and $ 1\rangle \rightarrow 0\rangle$	<code>circ.x(0)</code>
H	Puts a qubit into superposition	<code>circ.h(0)</code>
CX	Entangles two qubits (control + target)	<code>circ.cx(0, 1)</code>

You met these gates in Lab 0; here we build circuits with them in C++.

What's a Bell state?

A two-qubit **entangled** state. The two most common forms:

- $(|00\rangle + |11\rangle) / \sqrt{2}$: the qubits are perfectly correlated
- $(|01\rangle + |10\rangle) / \sqrt{2}$: the qubits are anti-correlated

Built with one `H` followed by one `CX` :

```
QuantumCircuit circ(2, 0);  
circ.h(0);  
circ.cx(0, 1);    // gives you |00> + |11>
```


Exercise 1: Build the other Bell state

Your task: produce $(|01\rangle + |10\rangle) / \sqrt{2}$.

You already know how to build $|00\rangle + |11\rangle$. What single gate gets you from one to the other?

Hint in the notebook. **Three** valid solutions exist.

Type your code in `bell_exercise.cpp`, then run the build/run cell and the grade cell.

Optional stretch: relative phase

Two more Bell states use a minus sign:

- $(|00\rangle - |11\rangle) / \sqrt{2}$
- $(|01\rangle - |10\rangle) / \sqrt{2}$

The Z gate introduces the minus sign (`circ.z(0)`). The notebook walks through it; no grading on this one.

Part 2: Circuit Depth

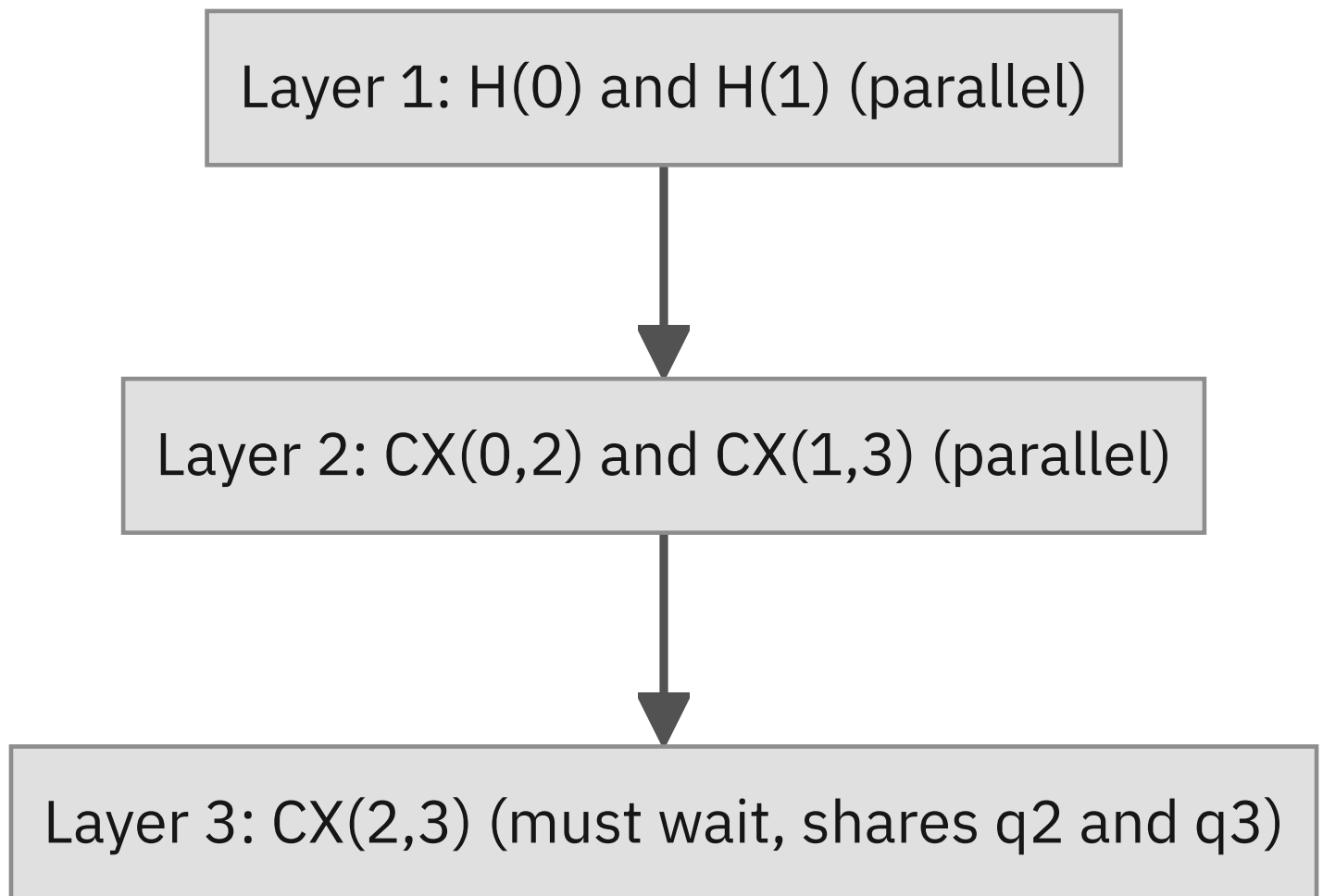
James Weaver, July 2026, Qiskit Global Summer School

11 / 29

What "depth" means

The number of **time steps** to run a circuit, assuming you run as many gates in parallel as possible.

Two gates can share a layer **only if** they touch completely different qubits.



Lower depth = less time for noise to accumulate = better results on hardware.

GHZ states: the N-qubit Bell state

$$|\text{GHZ}_N\rangle = (|00\dots 0\rangle + |11\dots 1\rangle) / \sqrt{2}$$

All N qubits agree: either all 0 or all 1.

The naive ways to build it (fan-out from qubit 0, or chain along the line) both produce **depth N**. For 100 qubits that's 100 layers. Way too noisy for hardware.

Exercise 2: Predict the depth

You'll see two small circuits. **Count the layers** by hand, then type your predictions into the grade cell:

```
grade_lab1_ex2({"a": your_answer_a, "b": your_answer_b})
```

Trap to watch for: gates can look parallel in the diagram but actually share a qubit. Trace the qubits, not the picture.

Partial credit if you get one right.

The recursive fan-out trick

What if every already-entangled qubit could entangle **one new qubit** per layer?

LAYER	ENTANGLED QUBITS
0	1
1	2
2	4
3	8
4	16

Each layer **doubles** the count. Depth grows as $\log_2(N)$ instead of N .

For 16 qubits: depth 5 instead of 16. For 1024 qubits: depth 11 instead of 1024.

Exercise 3: Depth-5 16-qubit GHZ

The notebook gives you the scaffolding:

```
QuantumCircuit circ(16, 0);

// Layer 0: Hadamard
circ.h(0);

// Layer 1: 1 CX gate -> 2 entangled qubits
// YOUR CODE HERE

// Layer 2: 2 CX gates -> 4 entangled qubits
// YOUR CODE HERE
// ...continues to Layer 4
```

Fill in **one layer's worth of CX gates** under each marker. The check prints Valid 16-qubit GHZ state with depth 5! if right.

Part 3: Transpilation

James Weaver, July 2026, Qiskit Global Summer School

17 / 29

Real hardware has rules

When you submit a circuit, the **transpiler** rewrites it to obey three constraints:

1. **Native gates only:** IBM Heron supports $\{R_Z, \sqrt{X}, X, CZ\}$. No H , no CX directly.
2. **Limited connectivity:** not every pair of qubits is physically connected.
3. **Physical qubit assignment:** your abstract qubit q_0 gets mapped to a real qubit.

Non-neighbor CX gates require **SWAP** gates (3 CZ each). That's the killer of naive designs.

The bridge identity

A clever way to do $CX(0, 2)$ on a chain 0-1-2 without SWAPs:

$$CX(0, 2) = CX(0,1) \cdot CX(1,2) \cdot CX(0,1) \cdot CX(1,2)$$

Read left to right in time: apply $CX(0,1)$ first, then $CX(1,2)$, then $CX(0,1)$ again, then $CX(1,2)$ again.

Four nearest-neighbor CX gates produce the exact same transformation as one long-distance $CX(0, 2)$. The middle qubit acts as a "bridge".

Cost: 4× as many CX gates as a single local one. Still better than 3 SWAPs.

Exercise 4: Use the bridge

Build a 3-qubit GHZ on a chain 0-1-2 using **only** nearest-neighbor CX gates. Start with:

```
QuantumCircuit qc(3, 0);  
qc.h(0);  
qc.cx(0, 1);
```

// YOUR CODE HERE: replace the non-local CX(0, 2) using the bridge identity

The grader checks two things:

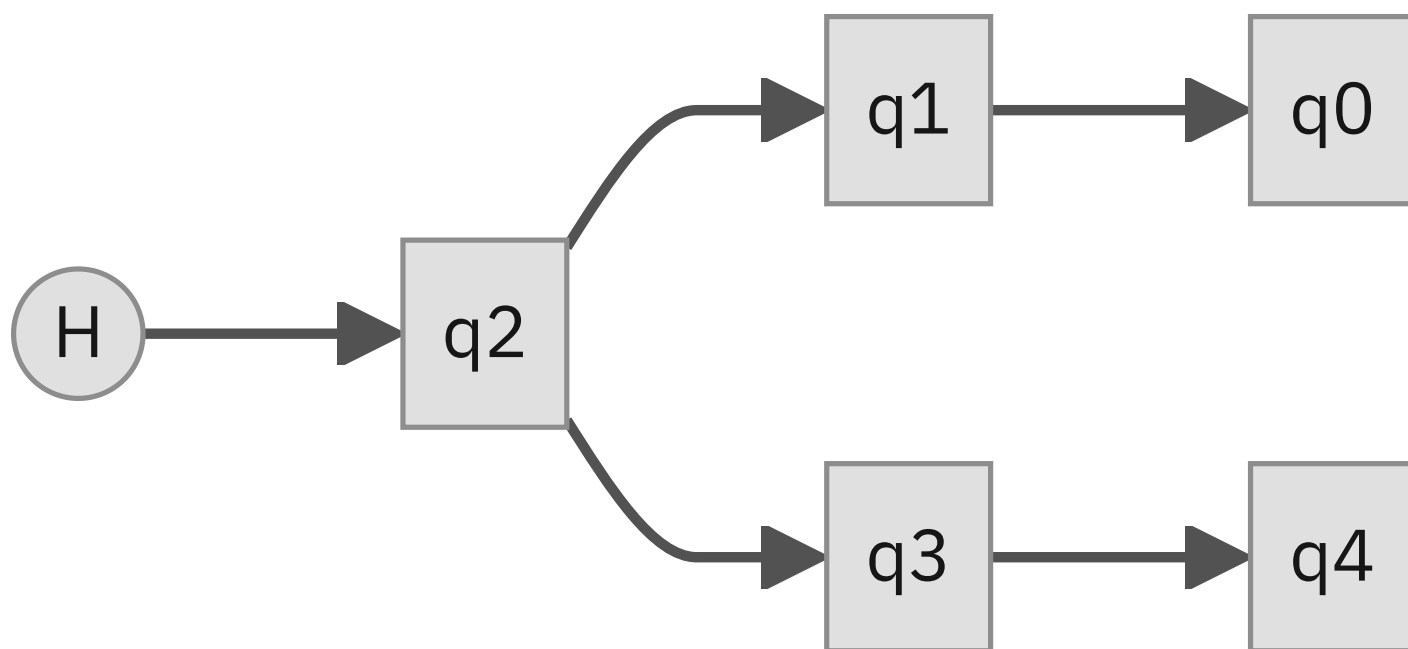
- The state is a valid 3-qubit GHZ
- **No CX skips a qubit** (no `CX(0, 2)`)

Start from the middle

For an N-qubit GHZ on a line:

- Fan-out **from the end** → depth N
- Fan-out **from the middle** → depth $\approx N/2$

Both halves of the line propagate entanglement in parallel:



Exercise 5: Depth-4 5-qubit GHZ

On the line 0-1-2-3-4, build a GHZ with abstract depth **4**.

```
QuantumCircuit circ(5, 0);
```

```
// YOUR CODE HERE: build a depth-4 GHZ starting from the middle
```

Two CX gates per layer can run in parallel if they touch disjoint qubits. Use that.

Heavy-hex topology

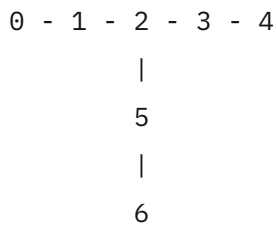
IBM Heron's qubits are arranged in a **heavy-hex** pattern:

- Most qubits have **2 neighbors** (degree 2): they sit on chains
- Some qubits have **3 neighbors** (degree 3): **junctions** where chains branch

Strategy: start your GHZ at a **junction** to get parallelism in three directions.

Exercise 6: 7-qubit T-shape GHZ

The subgraph is a horizontal chain with one branch downward:



Qubit 2 is the degree-3 junction. **Build a 7-qubit GHZ on this subgraph with minimum depth.**

Grader gives **2 points** for optimal depth (5), **1 point** for a correct but slower GHZ.

Exercise 7: The 64-qubit final challenge

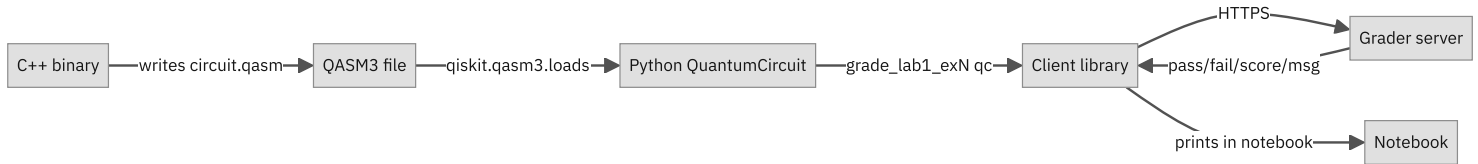
You'll use **BFS** (breadth-first search) to find a spanning tree of qubits 0 through 63 on FakeTorino, then build the GHZ along that tree.

```
for (int d = 1; d <= max_d; d++) {  
    for (int q = 0; q < n; q++) {  
        if (bfs_depth.count(q) && bfs_depth[q] == d) {  
            // YOUR CODE HERE: add CX from parent[q] to q  
        }  
    }  
}
```

You only fill in **one line**, but understanding what `parent[q]` means is the win.

How the grader works

The C++ binary writes your circuit as **OpenQASM 3** to a file. A Python grade cell reads it back, reconstructs a `QuantumCircuit`, and hands it to the **same qc_grader client and same grader server** the Python lab uses. Only the front of the pipeline is new.



- After each exercise's build cell, run the grade cell beneath it.
- You'll see 🎉 Correct! or a hint about what went wrong.
- **Partial credit** exists on Ex2 (1/2 if one right) and Ex6 (1/2 if suboptimal depth).

Tips for success

- **Run cells in order**, top to bottom; variables and files from earlier cells are needed later
- If a grade cell shows a stale result, **rebuild the exercise first** so the new `circuit.qasm` is written
- Use `circ.draw()` to **visualize** your circuit while debugging
- The `## Click to reveal` blocks in the notebook are there if you're truly stuck; **try first**, peek second
- Stuck on Ex7? Walk through `parent[q]` for `q=1`, `q=2`, `q=3` by hand

What you'll carry into Lab 2

- A real intuition for **circuit depth and noise**
- A working **64-qubit GHZ** circuit for the Heron architecture, built in C++
- The mental habit of **designing for the chip**, not against it

Lab 2 compares the Heron processor against IBM's new **Nighthawk** processor using GHZ benchmarks built on the same topology-aware ideas you practiced here.

Thank you

- Lab notebook: `qgss26-labs/lab-1/cpp/cpp_lab_1.ipynb`
- Grader: `grade_lab1_exN(...)` for each exercise (via QASM3 bridge)

Have fun. Circuit depth matters more than you think.